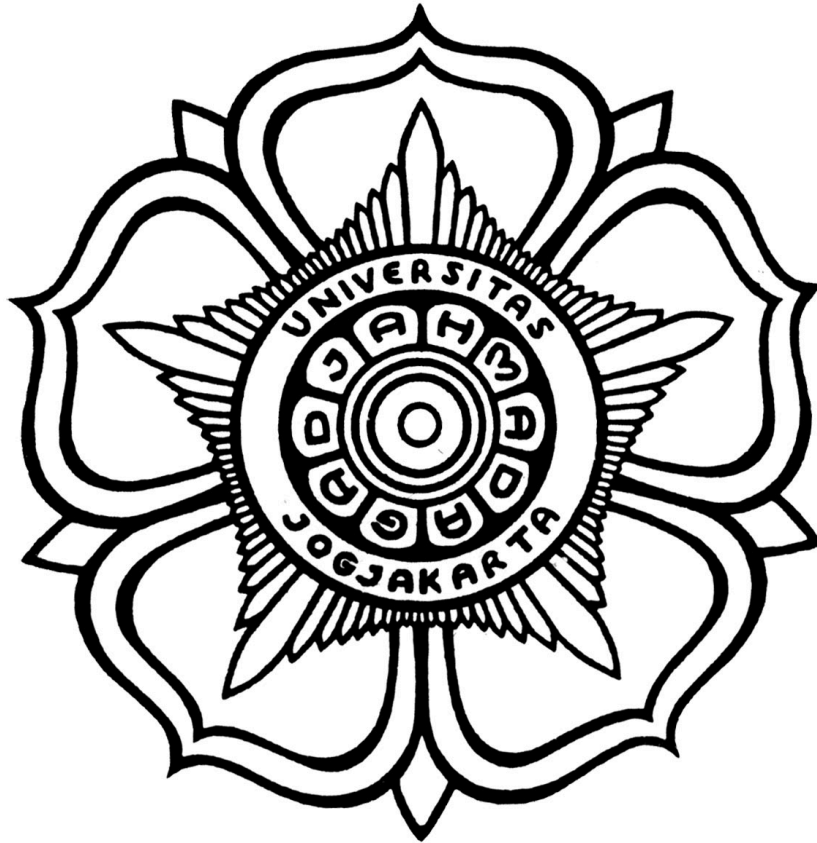


**CAMPUS EVENT MANAGEMENT SYSTEM
SIMAAUGM**



Members	: Matthew Harry Indriadi
(24/532723/PA/22527)	
	Farhan Adiwidya Pradana
(24/536804/PA/22773)	
	Christiano Jose Intoro
(24/546891/PA/23217)	
Course	: Database CS-A (MII212501)
Date	: 07/11//2025

I. INTRODUCTION

A. Background and Problem

Universitas Gadjah Mada frequently organizes a wide range of student events such as seminars, workshops, and competitions. However, the management of these events — from event creation and venue assignment to participant registration — is often carried out using separate spreadsheets or online forms. This fragmented approach leads to redundant data, inconsistent records, and difficulty in tracking attendance and event outcomes.

SIMAA UGM (Sistem Manajemen Acara UGM) is designed to address these issues by providing a centralized, structured, and efficient system for managing campus events. The system enables organizers to create and manage events, allows students to register for participation, and records venue details systematically, ensuring consistency and reliability of data within a unified database.

B. Objectives

The main objectives of SIMAA UGM are to:

- i. To design a simple yet structured database system for storing event and registration data.
- ii. To support essential CRUD operations for users, events, and registrations.
- iii. Maintain data integrity by linking event, venue, and registration records accurately.
- iv. Provide a scalable foundation for future enhancements such as feedback collection and reporting features.

C. Users and Roles

In this version, the system distinguishes between Students and Organizers as separate entities.

- Organizer: Responsible for creating and managing events, including setting details such as date, location, and capacity..
- Student: Views and registers for available events.

D. Scope

The initial version of SIMAA UGM focuses on three core functionalities:

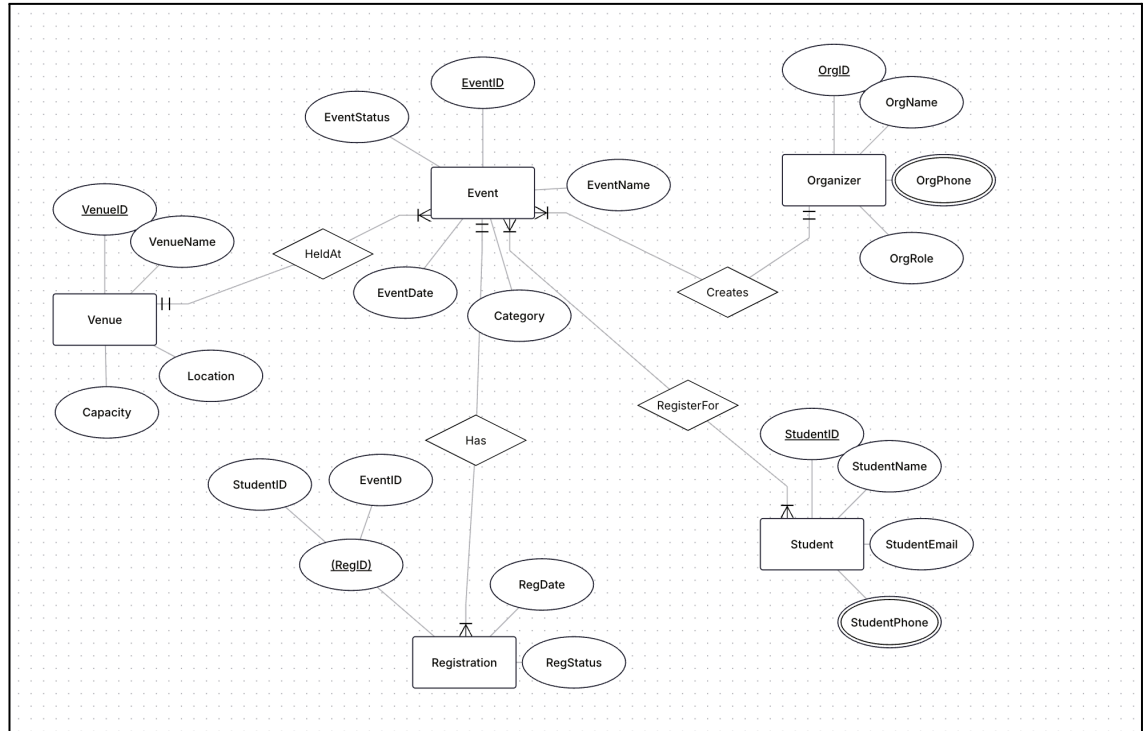
- i. Managing organizer and student information.
- ii. Creating and managing event details, including the assigned venue and event schedule.
- iii. Handling participant registration for events and maintaining registration status records.

Out of scope for this version: feedback management, payment systems, reporting modules, and analytics features (to be considered in future development).

E. Primary Use Cases

In the SIMAA UGM system, organizers create and manage events by specifying event details such as title, venue, schedule, and capacity. Students can browse available events, register to participate, and cancel their registrations when needed. Organizers can also monitor registration data to ensure effective event management and venue utilization.

II. DATABASE DESIGN



A. Entity Explanation

Entities:

- Student
 - Attributes: StudentID (primary key), StudentName, StudentEmail, StudentPhone
- Event
 - Attributes: EventID (primary key), EventName, EventStatus, Category, EventDate
- Organizer
 - Attributes: OrgID (primary key), OrgName, OrgPhone, OrgRole
- Venue
 - Attributes: VenueID (primary key), VenueName, Location, Capacity
- Registration

- Attributes: RegID (primary key), RegDate, RegStatus

Relationship:

Relationship	Entities Involved	Cardinality	Explanation
RegisterFor	Event ↔ Student	Many-to-Many (M:N)	One student can register for multiple events One event can have multiple student registrations
Creates	Organizer ↔ Event	One-to-Many (1:N)	One organizer can create multiple events One event is created by one organizer
Has	Event ↔ Registration	One-to-Many (1:N)	One event can have multiple registrations Each registration is for exactly one event
HeldAt	Event ↔ Venue	Many-to-One (N:1)	One event is held at one venue One venue can host multiple events (at different times)

B. Normalization Steps

- **Unnormalized Form**

→ **Table column :**

1. **EventID**
2. **EventName**
3. **EventDate**
4. **EventStatus**
5. **Category**
6. **VenueID**
7. **VenueName**
8. **Location**
9. **Capacity**
10. **OrgID**
11. **OrgName**
12. **OrgPhone**
13. **OrgRole**
14. **StudentID**
15. **StudentName**
16. **StudentEmail**
17. **StudentPhone**
18. **RegID**
19. **RegDate**
20. **RegStatus**

→ So there are 20 columns on 1 single table which represent the database

- **Problems:**
 - Repeating student data (if multiple students register for same event).
 - Repeating venue and organizer info for every event.
 - Data redundancy and update anomalies.
- **First Normal Form (1NF)**
 - Separate data into clear tables
 - **Student**

StudentID	StudentName	StudentEmail	StudentPhone
-----------	-------------	--------------	--------------

- **Event**

EventID	EventName	EventDate	EventStatus	Category	VenueID	OrgID
---------	-----------	-----------	-------------	----------	---------	-------

- **Registration**

StudentID	EventID	RegDate	RegStatus
-----------	---------	---------	-----------

- **Venue**

VenueID	VenueName	Location	Capacity
---------	-----------	----------	----------

- **Organizer**

OrgID	OrgName	OrgPhone	OrgRole
-------	---------	----------	---------

- **Second Normal Form (2NF)**

→ Ensure all non-key attributes depend on the whole primary key, not part of it.

→ In our case:

- Each table already has a single primary key (EventID, StudentID, etc.),
- Registration uses RegID as a primary key, not a composite (StudentID, EventID),

→ so there's no partial dependency (No changes needed)

- **Third Normal Form (3NF)**

→ Remove transitive dependencies (non-key depends on another non-key):

- a. In Event, all non-key attributes depend only on EventID.
- b. In Venue, VenueName, Location, Capacity depend only on VenueID.
- c. In Organizer, same rule. In Student, all depend on StudentID.
- d. In Registration, all depend on RegID.

→ No transitive dependencies remain.